

Blender Helper

Полная документация и инструкция по разворачиванию

Blender Helper — полная документация и инструкция по разворачиванию

Этот документ — две вещи в одном: (1) полное описание того, что такое бот Blender Helper и что он умеет, и (2) пошаговая инструкция «от идеи до автономного Telegram-бота на бесплатном сервере», написанная по следам реального прохождения всего пути, включая все ошибки, на которые наступали, и как их обойти сразу, не повторяя их.

Бот — не обёртка над ChatGPT/Gemini и не пытается ей быть. Это бесплатная экспертная система по Blender: работает без платного AI API, ищет ответы в собственной курируемой базе знаний (официальный Blender Manual + проверенные личные заметки), понимает версии Blender, честно говорит «не знаю», когда данных недостаточно, и умеет вести диагностику проблем и обучение — а не просто отвечать на всё подряд как общий чат-бот. Полное техническое задание — `docs/Blender_Expert_System_v2_TZ.pdf`, журнал того, как и почему всё было построено именно так — `PROJECT_PLAN.md`.

Оглавление

Часть 1 — что за бот

1. Что это за бот
2. Что бот умеет
3. Как устроена база знаний
4. Архитектура кода

Часть 2 — как развернуть

5. Что нужно заранее
6. Создание бота в Telegram
7. Локальный запуск на Windows (для разработки)
8. Создание бесплатного сервера в Oracle Cloud
9. Подключение к серверу по SSH
10. Установка Python и окружения на сервере
11. Разворачивание бота на сервере (первый деплой)
12. Автозапуск через systemd
13. Обновление уже развёрнутого бота
14. Указатель по официальному руководству Blender
15. Шпаргалка команд на каждый день
16. Сводка всех проблем и решений

1. Что это за бот

Blender Helper (@Blenderhelpbot) — Telegram-бот-помощник по программе Blender (3D-графика, моделирование, анимация, рендеринг). Изначально был простым FAQ-ботом с парой десятков

вопрос-ответов; переделан в полноценную экспертную систему по жёсткому техническому заданию (15 фаз, раздел 40 ТЗ, см. PROJECT_PLAN.md — там журнал каждой фазы с отчётом «что изменено, что добавлено, что не сделано и почему»).

Жёсткие принципы, заложенные в архитектуру с самого начала (раздел 2, 26, 44 ТЗ):

- Без обязательного внешнего AI API. Никакого OpenAI/Anthropic/Gemini

под капотом. Поиск — собственный TF-IDF (раздел 10 ТЗ), написанный вручную на стандартной библиотеке (без numpy/scipy — сервер слабый, бесплатный тариф Oracle Cloud).

- Честность важнее уверенности (Zero-Hallucination Mode, раздел 26 ТЗ).

Если бот не уверен — он либо переспрашивает («Возможно, ты имел в виду?»), либо прямо говорит, что не нашёл ответа. Не выдумывает технические детали из догадки.

- Источники разного качества ранжируются, а не смешиваются.

Официальный Blender Manual считается надёжнее личных заметок — при конфликте побеждает более авторитетный источник (раздел 3, 17 ТЗ).

- Не пытается симулировать ChatGPT (раздел 44 ТЗ). Это специализированная система про Blender с курируемой базой знаний, а не бот с общими знаниями обо всём на свете. На сложные составные вопросы, которых нет в базе, он честно не ответит — это ожидаемое поведение, а не баг (подробнее — раздел 3 ниже, «Пределы возможностей»).

Бот развёрнут на Oracle Cloud Free Tier (бесплатный сервер, работает 24/7 независимо от чьего-либо компьютера) и обслуживает реальных пользователей в Telegram.

2. Что бот умеет

2.1 Обычные вопросы про Blender

Напиши боту вопрос текстом — он ищет ответ по всей базе знаний (официальный Manual + личные заметки), учитывая термины, синонимы и разные формы русских слов (см. раздел 3). Обычный ответ — просто текст, без служебных пометок:

Вы: Что такое модификаторы? Бот: Модификаторы — это автоматические операции, которые неразрушающим образом влияют на геометрию объекта...

Если бот не до конца уверен, что нашёл именно то, что нужно, — переспросит с кнопками «Да, это оно» / «Нет», а не ответит наугад.

Просьба рассказать подробнее. После любого уверенного ответа можно попросить больше информации — бот помнит только ПОСЛЕДНИЙ заданный вопрос (не всю переписку) и по фразам вроде «подробнее», «ещё информация», «дай источники», «покажи ссылки», «где почитать», «tell me more» подтянет дополнительные найденные материалы и список из нескольких источников со ссылками:

Вы: Что такое модификаторы? Бот: *(ответ)* Вы: подробнее Бот: Вот что ещё нашлось по теме: ...
Источники: • Введение, официальный Blender Manual (5.1) — <https://docs.blender.org/>... • ...

Если написать «подробнее» без предыдущего вопроса (например, в новом диалоге) — бот честно скажет, что не помнит контекст, и попросит задать вопрос целиком.

Пределы возможностей. База знаний курируемая и НЕ бесконечная — бот не заменяет ChatGPT/Gemini для сложных составных вопросов («в чём опасность делать X, если не сделано Y» и подобные). Если нужного ответа нет — он честно предложит переформулировать вопрос

покороче или заглянуть в `/hotkeys`, `/resources`, официальную документацию, а не придумает правдоподобный, но непроверенный ответ.

2.2 Диагностика проблем

Для вопросов вида «после Subdivision модель ломается» или «рендер чёрный» бот не вываливает сразу список из десяти возможных причин, а задаёт по одному наиболее информативному уточняющему вопросу за раз (decision-tree диалог, раздел 12-13 T3) и постепенно сужает причину:

Вы: После Subdivision модель ломается Бот: Где именно появляется проблема? [На углах] [На отверстиях] [На плоских поверхностях] [По всей модели] *(после выбора — следующий уточняющий вопрос или готовое решение)*

Если написать что-то текстом вместо клика по кнопке — диалог диагностики считается брошенным, начинается заново со следующего сообщения.

2.3 Обучение

- `/learn <тема>` — теория и практический пример по теме (например, `/learn Mirror Modifier`)
- `/test` — проверочные вопросы по последней изученной теме
- `/exam` — вопросы вперемешку по всем темам
- `/next` — следующий вопрос теста или подсказка, что изучить дальше
- `/progress` — прогресс: сколько тем пройдено, сколько вопросов отвечено

верно

- `/weaknesses` — слабые места (темы, где чаще всего ошибались)

Прогресс хранится в SQLite и переживает перезапуск бота — можно продолжить с того же места даже после обновления кода на сервере.

2.4 Справочные команды

- `/start` — приветствие и краткая справка (подписывает на рассылку)
- `/help` — список команд (то же приветственное сообщение)
- `/hotkeys` — горячие клавиши по категориям (навигация, выделение, Edit Mode, модификаторы, Sculpt, шейдинг, UV, анимация и т.д.)
- `/resources` — сайты с 3D-моделями, текстурами, HDRI, аддонами и обучающими материалами
- `/news` — свежие новости из мира CG (BlenderNation, 80 Level, CG Channel)
- Inline-режим — вызов бота из любого чата командой ``@Blenderhelpbot вопрос``, без добавления бота в этот чат

2.5 Команды владельца бота (администрирование)

Всё ниже доступно ТОЛЬКО тому, чей Telegram ID указан в `.env` как `OWNER_ID` — остальным пользователям бот вежливо откажет.

- `/broadcast <текст>` — рассылка сообщения всем, кто хоть раз запускал

/start

- /admin — список всех admin-команд прямо в Telegram
- /health — жива ли база знаний, аптайм процесса, счётчики основных реестров (сколько chunks, терминов, диагностических проблем, уроков)
- /stats — подробная статистика: chunks по типу источника и уровню авторитетности, число терминов/хоткеев/уроков, подписчиков рассылки, профилей пользователей, незнакомых вопросов в логе
- /sources — какие файлы подключены как источник поиска и сколько chunks дал каждый
- /version — какие версии Blender реально покрыты базой знаний (не версия самого бота)
- /search <запрос> — сырые результаты поиска с полным разложением score (для отладки ранжирования)
- /debug <вопрос> — полный разбор одного вопроса: распознанный intent, тема, версия, сработала ли диагностика, выбранный источник, confidence — то, что скрыто от обычных пользователей (раздел 3.3 ТЗ, Debug Mode)
- /reindex — перечитать базу знаний, диагностические деревья и уроки с диска БЕЗ перезапуска процесса — полезно после ручного редактирования knowledge/*.json на сервере, если не хочется дожидаться systemctl restart

3. Как устроена база знаний

На момент написания этого документа (см. /stats для актуальных цифр — база растёт со временем):

Источник	Что это	Объём
Официальный Blender Manual 5.1	Переведённые фрагменты официальной документации	~770 chunks
Личные заметки (knowledge/personal/dima_notes)	Проверенные вручную короткие ответы на конкретные вопросы, помечены как «личная база, не проверено официально» при конфликте с Manual	~84 записи
Терминология (knowledge/system/terminology)	Двуязычные термины (английское/русское название, алиасы, UI label) — чтобы поиск понимал «Mirror», «зеркало» и «модификатор зеркала» как одно и то же	38 терминов
Диагностические деревья (knowledge/system/diagnostics)	Decision-tree сценарии для типовых проблем	2 дерева (Subdivision ломает модель, чёрный рендер/материал)
Уроки (knowledge/system/education)	Теория + пример + тест	3 урока (Mirror, Boolean, N-gon)
Горячие клавиши (data/hotkeys.json)	Отдельный точный справочник, не через поиск по смыслу	93 клавиши, 11 категорий

Почему объём такой, а не «все вопросы о Blender»: раздел 36 ТЗ прямо формулирует принцип — «1000 качественных chunks лучше 100000 мусорных». Каждая единица контента либо взята из официального источника, либо вручную проверена. Это осознанный компромисс между полнотой и достоверностью, а не недоработка.

Поиск понимает разные формы слов. С последней правки бот приводит русские слова к начальной форме перед сравнением (лемматизация, рунорфyz) — «фаску»/«фаски», «применён»/«применить», «модификаторы»/«модификатор» считаются одним и тем же словом. Понимает и смешанный русский/английский ввод, и алиасы/синонимы терминов.

4. Архитектура кода

```
blender_bot/
├── bot.py                # совместимый entry point для systemd (ExecStart всё ещё зовёт его)
├── app/main.py           # настоящая точка входа: сборка Application, регистрация хендлеров
├── bot/handlers/         # Telegram-слой: только формат сообщений и диалоги,
│   │                     без бизнес-логики поиска/диагностики
│   ├── qa.py             # обычные вопросы → search/qa_service.py
│   ├── diagnostics.py    # decision-tree диалоги
│   ├── education.py      # /learn /test /exam /progress /weaknesses /next
│   ├── admin.py          # /admin /health /stats /sources /version /search /debug /reindex
│   └── hotkeys.py, resources.py, news.py, start.py, broadcast.py, inline.py
├── knowledge/            # база знаний: official/, personal/, system/ (terminology,
│   │                     diagnostics, education), registry.py, schema.py, version.py
├── search/               # SearchEngine (TF-IDF + лемматизация + термины + версии +
│   │                     авторитетность), QAService (бизнес-логика ответа), confidence.py
├── intents/              # классификация типа вопроса (WHAT_IS/HOW_TO/...) и темы
├── diagnostics/          # реестр и логика decision-tree
├── education/            # реестр уроков
├── profile/              # SQLite user profile + подписчики рассылки
├── config/               # пути, токены, ID владельца (.env)
├── data/                 # hotkeys.json, resources.json, news_feeds.json (+ служебные логи)
├── tests/                # 236 юнит-тестов + tests/quality (540-кейсовый Quality Score suite)
├── scripts/              # разовые/обслуживающие скрипты (сборка/пересборка базы)
└── docs/Blender_Expert_System_v2_TZ.pdf # полное ТЗ
```

Полная карта с обоснованием каждого решения — CLAUDE.md (архитектурные правила) и PROJECT_PLAN.md (журнал по фазам, включая честный список известных ограничений на каждом этапе).

5. Что нужно заранее

- Компьютер с Windows
 - Аккаунт Telegram
 - Email для регистрации в Oracle Cloud (плюс банковская карта — для верификации личности, деньги с бесплатного тарифа не списываются)
 - GitHub-аккаунт, если код бота хранится там
-

6. Создание бота в Telegram

1. В Telegram напишите [@BotFather](#)
 2. Отправьте `/newbot`, следуйте инструкциям (имя бота, username)
 3. BotFather пришлёт токен вида `123456789:AAExampleTokenFromBotFather` — сохраните его, он понадобится дальше. Токен — секретная вещь, как пароль: у кого он есть, тот полностью управляет ботом.
 4. (Опционально, но полезно) В BotFather откройте Bot Settings → Inline Mode, включите и задайте текст-подсказку — это позволит вызывать бота из любого чата командой `@ИмяБота текст`, без добавления бота в этот чат.
 5. Узнайте свой Telegram user ID через [@userinfobot](#) — обязательно нужен, чтобы `.env` дал вам доступ к `/broadcast` и ко всем admin-командам из раздела 2.5 (`/admin`, `/stats`, `/debug` и т.д.).
-

7. Локальный запуск на Windows (для разработки)

1. Установите Python с [python.org](#) — при установке обязательно отметьте "Add python.exe to PATH"

Если у вас Windows поставила совсем свежую версию Python (например 3.14) — заранее знайте: некоторые библиотеки (в частности `python-telegram-bot`) могут упасть с ошибкой `RuntimeError: There is no current event loop`. Решение — либо `pip install --upgrade python-telegram-bot` (часто помогает), либо поставить дополнительно Python 3.12 через `py -3.12` и создавать `venv` именно на нём. Подробности — проблема №1 в разделе 16.

2. Откройте терминал в папке проекта: в Проводнике зайдите в папку, в адресной строке наберите `cmd`, нажмите Enter
3. Создайте виртуальное окружение и установите зависимости:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```
4. Скопируйте `.env.example` в `.env`, впишите реальный токен и свой Telegram ID (`OWNER_ID`) — без него не заработают `/broadcast` и все admin-команды из раздела 2.5
5. Запустите:

```
python bot.py
```

Строка `Бот запущен в терминале` — значит всё работает. Проверьте в Telegram командой `/start`.

Это годится для разработки и тестов, но бот работает только пока открыт терминал. Дальше — как разместить его на сервере, чтобы он работал сам по себе.

8. Создание бесплатного сервера в Oracle Cloud

Используем Oracle Cloud Free Tier — там есть по-настоящему бесплатный навсегда сервер (Always Free), которого достаточно для одного небольшого бота.

8.1 Регистрация

1. Зайдите на [oracle.com/cloud/free](#) →

Start for free

2. Зарегистрируйтесь (email, телефон, карта для верификации — не списывает деньги с Always Free ресурсов)

8.2 Создание сервера (Compute Instance)

В консоли: значок «гамбургер»-меню в левом верхнем углу → Compute → Instances → Create Instance.

Basic information:

- Name — можно оставить автоматическое, роли не играет
- Image — нажмите Change image, выберите Ubuntu (последняя LTS-версия, например 20.04 или 24.04)

Shape:

- Нажмите Change shape, выберите вариант с пометкой Always Free-eligible (обычно VM.Standard.E2.1.Micro)

Security:

- Ничего не меняйте, оставьте значения по умолчанию (Shielded instance выключен)

Networking — самый проблемный шаг, читайте внимательно:

1. Primary network → переключите на Create new virtual cloud network (если у вас ещё нет ни одной сети — это нормально для первого сервера)
2. Subnet → переключите на Create new public subnet (обязательно public)
3. Прокрутите вниз до Public IPv4 address assignment → включите

Automatically assign public IPv4 address

Известный баг: этот переключатель у нас несколько раз не включался (оставался серым), несмотря на то, что public subnet был выбран верно. Это не ошибка пользователя, а глюк интерфейса Oracle. Если не включается — не тратьте на это время, создавайте сервер как есть, без публичного IP. Добавим его отдельно после создания сервера (шаг 8.4 ниже) — это работает надёжнее, чем сам мастер создания.

Add SSH keys:

- Оставьте выбранным Generate a key pair for me
- Нажмите Download private key — скачается файл .key. **Сохраните его в отдельную, легко находимую папку** (например C:\oracle-vps\) — без него доступ к серверу восстановить нельзя

Storage:

- Ничего не меняйте, оставьте значения по умолчанию

Нажмите Create. Через 1-2 минуты статус сервера станет Running.

8.3 Если публичный IP не появился при создании

На странице сервера откройте вкладку Сети (Networking). Если поле

Общедоступный адрес IPv4 пустое — там же должен быть блок

Экспресс-действия → «Подключение общедоступной подсети к Интернету» с кнопкой Подключение. Нажмите её — Oracle подключит интернет-шлюз и настроит таблицу маршрутизации автоматически.

Если экспресс-действия нет или не помогло — добавьте IP вручную:

1. Вкладка Сети инстанса → нажмите на имя VNIC
2. Вкладка Администрирование IP-адресов
3. Найдите приватный адрес → значок □ (три точки) → редактировать
4. Тип общедоступного IP-адреса → выберите ****Динамический общедоступный IP-адрес → Обновить****

После этого на вкладке инстанса появится публичный IP — запишите его, он понадобится для подключения по SSH.

9. Подключение к серверу по SSH

1. В Проводнике откройте папку со скачанным .key-файлом, в адресной строке наберите `cmd`

2. Подключитесь:

```
ssh -i ssh-key-ИМЯФАЙЛА.key ubuntu@ВАШ_ПУБЛИЧНЫЙ_IP
```

(пользователь именно `ubuntu`, если ставили образ Ubuntu)

3. На вопрос про fingerprint ("Are you sure you want to continue connecting?")

ответьте `yes`

Частая ошибка на Windows: UNPROTECTED PRIVATE KEY FILE. Windows по умолчанию даёт слишком открытые права на файл ключа. Лечится так (в обычном Windows-терминале, не на сервере): ``
`icacls ssh-key-ИМЯФАЙЛА.key /inheritance:r icacls ssh-key-ИМЯФАЙЛА.key /grant:r "%username%":R`
` Если после этого ошибка осталась – проверьте, что реально удалось убрать: ` `icacls ssh-key-ИМЯФАЙЛА.key` ` Если в списке остались BUILTIN\Administrators, NT AUTHORITY\SYSTEM, Authenticated Users или BUILTIN\Users – уберите их явно: ` `icacls ssh-key-ИМЯФАЙЛА.key /remove:g "BUILTIN\Administrators" icacls ssh-key-ИМЯФАЙЛА.key /remove:g "NT AUTHORITY\SYSTEM" icacls ssh-key-ИМЯФАЙЛА.key /remove:g "NT AUTHORITY\Authenticated Users" icacls ssh-key-ИМЯФАЙЛА.key /remove:g "BUILTIN\Users"` `` Должна остаться только ваша учётная запись. Попробуйте подключаться заново.

Если SSH неожиданно обрывается (`client_loop: send disconnect: Connection reset`) — это иногда бывает, особенно при первом подключении. Просто подключайтесь заново той же командой `ssh -i`
....

Если увидели приглашение вида `ubuntu@instance-...:~$` — вы внутри сервера.

10. Установка Python и окружения на сервере

10.1 Обновление системы и git

```
sudo apt update && sudo apt upgrade -y  
sudo apt install -y git
```


10.2 Установка Python 3.11 через pyenv

Проверьте версию системного Python:

```
python3 --version
```

Ubuntu 20.04 обычно ставит Python 3.8 — этого мало для современных версий python-telegram-bot (нужен минимум 3.9-3.10, лучше 3.11).

Не тратьте время на deadsnakes PPA. Стандартный совет в интернете — поставить нужную версию Python через `add-apt-repository ppa:deadsnakes/ppa`. У нас этот путь трижды ломался (пакет "находился" в репозитории, но `apt install` не мог его увидеть — кэш `apt` не докачивал список пакетов). Чинится через `sudo rm -rf /var/lib/apt/lists/* && sudo apt update`, но результат нестабилен. Сразу используйте `pyenv` — он не зависит от репозитория Ubuntu и просто компилирует Python из исходников.

Установите инструменты для сборки и сам `pyenv`:

```
sudo apt install -y build-essential zlib1g-dev libncurses5-dev libgdbm-dev libnss3-dev libssl-dev libreadline-dev
sudo apt install -y screen
curl -fsSL https://pyenv.run | bash
```

Добавьте `pyenv` в переменные окружения — одной командой, не по одной строке через несколько `echo`:

Если писать три отдельные команды `echo '...' >> ~/.bashrc` одну за другой — при вставке в терминал строки могут "слипнуться" без переноса (например, `~/.bashrc`echo), и `.bashrc` испортится с ошибкой синтаксиса при следующем открытии терминала. Используйте один `printf`:

```
printf '\nexport PYENV_ROOT="$HOME/.pyenv"\n[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"\n\nsource ~/.bashrc\n\npyenv --version\n'
```

Если `.bashrc` всё же оказался повреждён (ошибка `syntax error near unexpected token` при `source`), почистите дубликаты и пересоздайте блок:

```
sed -i '/PYENV_ROOT/d; /pyenv init/d; /pyenv virtualenv-init/d' ~/.bashrc
printf '\nexport PYENV_ROOT="$HOME/.pyenv"\n[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"\n\nsource ~/.bashrc\n'
```

10.3 Компиляция Python — обязательно внутри `screen`

Компиляция Python на слабом сервере занимает 10-15 минут. Если за это время оборвётся SSH-соединение (а на первых порах оно обрывается часто) — процесс компиляции умрёт вместе с закрытым терминалом, и всё придётся начинать заново. Чтобы это не повторилось — обязательно запускайте установку внутри `screen`, программы, которая держит процессы живыми на сервере независимо от вашего SSH-окна.

```
screen -S pyenv_install
pyenv install 3.11.9
```

Дождитесь строки `Installed Python-3.11.9 to /home/ubuntu/.pyenv/versions/3.11.9`.

Если SSH оборвётся во время ожидания — просто переподключитесь и вернитесь к сессии:

```
ssh -i ssh-key-ИМЯФАЙЛА.key ubuntu@ВАШ_IP
screen -r pyenv_install
```

Если пишет `There is no screen to be resumed... (Attached)` — добавьте флаг `-d`, который принудительно отключает старое мёртвое подключение:

```
screen -d -r pyenv_install
```

После успешной установки:

```
pyenv global 3.11.9
python3 --version
```

Должно показать Python 3.11.9. Выйдите из screen обычной командой `exit`.

11. Разворачивание бота на сервере (первый деплой)

11.1 Скачать код

```
git clone -b ИМЯ_ВЕТКИ https://github.com/ВАШ_АККАУНТ/ВАШ_РЕПОЗИТОРИЙ.git
cd ВАШ_РЕПОЗИТОРИЙ/blender_bot
```

11.2 Виртуальное окружение и зависимости

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

В начале строки терминала должно появиться `(venv)`.

Зависимости включают `ru morphology3` (лемматизация русских словоформ, раздел 3 выше) — при установке скачивает языковой словарь (`ru morphology3-dicts-ru`), это увеличивает время `pip install` и первый старт бота на несколько секунд (словарь грузится в память один раз при старте). Если когда-нибудь в старом окружении стоял `ru morphology2` (не `ru morphology3`) — он не заработает на современном Python (см. проблему №11 в разделе 16), нужен именно `ru morphology3`.

11.3 Настройка `.env`

```
nano .env
```

Впишите (замените на реальные значения):

```
BOT_TOKEN=ваш_настоящий_токен_от_botfather
OWNER_ID=ваш_telegram_id
```

`OWNER_ID` — не опционально, если хотите пользоваться `/broadcast` и ЛЮБОЙ из `admin`-команд раздела 2.5 (`/admin`, `/health`, `/stats`, `/sources`, `/version`, `/search`, `/debug`, `/reindex`) — без него все они откажут с сообщением «доступно только владельцу».

Сохраняйте строго в этом порядке:

1. `Ctrl+O`, затем `Enter` (сохранить)
2. Только затем `Ctrl+X` (выйти)

Если закрыть терминал/окно раньше, чем нажать `Ctrl+O` → `Enter` → `Ctrl+X` — файл не сохранится. Обязательно проверяйте после сохранения: `cat .env` — должны увидеть обе строки. Если `cat: .env: No such file or directory` — значит не сохранилось, повторите `nano .env` заново.

Если при повторном открытии `nano` пишет `File .env is being edited by ubuntu (with nano ..., PID ...)`; `open anyway?` — это безобидное предупреждение об оставшемся "зависшем" процессе от прошлой попытки. Смело жмите `Y`.

11.4 Тестовый запуск

```
python bot.py
```

Строка Бот запущен и Application started — значит всё работает. Проверьте в Telegram: /start, затем /admin (если указали свой OWNER_ID) — должен показать список admin-команд. Остановите тестовый запуск клавишами Ctrl+C перед следующим шагом.

12. Автозапуск через systemd

Это финальный шаг, который делает бота по-настоящему автономным — он будет работать даже после закрытия терминала и перезагрузки сервера.

```
sudo nano /etc/systemd/system/blenderbot.service
```

Вставьте (замените пути на свои, если отличаются — узнать текущий путь можно командой pwd, находясь в папке бота):

```
[Unit]
Description=Blender Telegram Bot
After=network.target
```

```
[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/ВАШ_РЕПОЗИТОРИЙ/blender_bot
ExecStart=/home/ubuntu/ВАШ_РЕПОЗИТОРИЙ/blender_bot/venv/bin/python /home/ubuntu/ВАШ_РЕПОЗИТОРИЙ/blender_bot/bot.py
Restart=always
RestartSec=5
```

```
[Install]
WantedBy=multi-user.target
```

Сохраните: Ctrl+O → Enter → Ctrl+X. Проверьте содержимое (cat /etc/systemd/system/blenderbot.service) — так же, как с .env.

Включите и запустите службу:

```
sudo systemctl daemon-reload
sudo systemctl enable blenderbot
sudo systemctl start blenderbot
sudo systemctl status blenderbot
```

Должно быть Active: active (running) и enabled. Готово — можно закрывать терминал и выключать компьютер, бот продолжит работать.

13. Обновление уже развёрнутого бота

Если бот уже работает на сервере (в проде, с реальными пользователями), а вы обновили код локально — это не «первый деплой» из раздела 11, а более осторожная процедура. Ниже — реально применённый порядок (использовался для перевода бота со старой дореформенной архитектуры на текущую, все шаги проверены на живом сервере, не только в теории).

13.1 Перед началом — бэкап

Живой сервис ещё не тронут, это чисто подготовительный шаг:

```
cd ~
mkdir -p backups
tar --exclude=ВАШ_РЕПОЗИТОРИЙ/blender_bot/venv --exclude=ВАШ_РЕПОЗИТОРИЙ/blender_bot/__pycache__ \
    -czf backups/blender_bot_pre_update_$(date +%Y%m%d_%H%M%S).tar.gz -C ВАШ_РЕПОЗИТОРИЙ blender_bot
```

У tar флаги --exclude должны идти ДО позиционных аргументов (пути к файлам), иначе будет ошибка options were used after any non-optional arguments.

13.2 Проверить состояние репозитория на сервере

```
cd ~/ВАШ_РЕПОЗИТОРИЙ/blender_bot
git status --short
```

Если пусто (чистое дерево, без локальных правок) — можно спокойно обновляться. Если что-то показывает — это чьи-то незакоммиченные правки прямо на сервере (например, ручное редактирование knowledge/*.json через nano); сначала решите, сохранять их или отбросить, не затирайте вслепую.

13.3 Обновить код

```
git fetch origin
git pull --ff-only origin ИМЯ_ВЕТКИ
```

--ff-only — сознательно, а не просто git pull: если сервер каким-то образом разошёлся с origin (кто-то закоммитил прямо на сервере), команда честно откажется вместо того, чтобы тихо создать merge-коммит или конфликт посреди прод-окружения. Если --ff-only отказал — разбирайтесь руками (git log, git diff), не форсируйте.

Если менялись зависимости (requirements.txt) — обновите venv:

```
source venv/bin/activate
pip install -r requirements.txt
```

13.4 Прогнать тесты прямо на сервере

Не только локально — окружение на сервере (версия Python, пакетов) может отличаться:

```
python -m unittest discover tests
```

Ожидается ok и Quality Score в конце вывода. Если тесты падают —

не перезапускайте сервис, разбирайтесь с причиной сначала (старый процесс всё ещё работает на старом коде, пользователи не пострадают, пока вы не сделали restart).

13.5 Перезапустить и проверить

```
sudo systemctl restart blenderbot
sleep 3
sudo systemctl status blenderbot
```

Ожидается Active: active (running). Затем проверьте логи на чистый старт без traceback:

```
sudo journalctl -u blenderbot --since '30 seconds ago' --no-pager
```

Ищите строки app.main - INFO - Бот запущен и Application started. Если видите traceback — читайте причину, при необходимости откатывайтесь (раздел 13.6).

Финальная проверка — не только по логам, а живым сообщением в Telegram (например, /start, затем обычный вопрос) — логи показывают, что процесс не упал, но не показывают, что бот отвечает ПРАВИЛЬНО.

13.6 Если что-то пошло не так — откат

```
sudo systemctl stop blenderbot
cd ~/ВАШ_РЕПОЗИТОРИЙ
```

```
git log --oneline -5          # найти коммит, на котором было всё хорошо
git reset --hard СТАРЫЙ_КОММИТ
sudo systemctl start blenderbot
```

Либо распаковать бэкап поверх (`tar -xzf backups/... -C ~/ВАШ_РЕПОЗИТОРИЙ`), если откат по `git` недостаточен (например, откатились данные, не только код).

14. Указатель по официальному руководству Blender

Исторический механизм из ранних версий проекта: скрипт `scripts/build_manual_index.py` клонирует открытый исходник документации (`projects.blender.org/blender/blender-manual`, лицензия CC-BY-SA), достаёт заголовок и краткое описание каждой страницы, переводит на русский и сохраняет в `data/manual_index.json`.

Важно: с Phase 7 (см. `PROJECT_PLAN.md`) этот файл живым поиском (`search/engine.py`) больше НЕ читается — единственный источник поиска теперь `knowledge/` (см. раздел 3-4 выше), включая уже проиндексированный и очищенный `knowledge/official/manual/5.1/manual.json`. `data/manual_index.json` остаётся на диске как исторический артефакт (в `.gitignore`, не в `git`), не мешает и не требуется для нормальной работы бота — этот раздел оставлен для полноты картины, повторять эти шаги для нового сервера не обязательно.

15. Шпаргалка команд на каждый день

Подключение к серверу:

```
ssh -i ssh-key-ИМЯФАЙЛА.key ubuntu@ВАШ_IP
```

Статус бота:

```
sudo systemctl status blenderbot
```

Перезапуск (например, после обновления кода):

```
sudo systemctl restart blenderbot
```

Остановка:

```
sudo systemctl stop blenderbot
```

Живые логи бота:

```
sudo journalctl -u blenderbot -f
```

(выход — `Ctrl+C`; чтобы не тонуть в шуме от постоянных запросов `getUpdates`, полезно `sudo journalctl -u blenderbot -f | grep -v getUpdates`)

Обновление кода бота на сервере (короткая версия, см. раздел 13 для полной осторожной процедуры):

```
cd ~/ВАШ_РЕПОЗИТОРИЙ/blender_bot
git pull --ff-only origin ИМЯ_ВЕТКИ
sudo systemctl restart blenderbot
```

Через Telegram (только для владельца, `OWNER_ID` в `.env`):

- `/health` — жива ли база знаний прямо сейчас
- `/stats` — актуальные цифры по базе знаний и пользователям

- `/debug <вопрос>` — почему бот ответил именно так (или не ответил)
- `/reindex` — пересчитать базу знаний без перезапуска процесса

16. Сводка всех проблем и решений

№	Проблема	Причина	Решение
1	<code>RuntimeError: There is no current event loop</code> при запуске бота на Windows	Слишком новая версия Python (3.14) несовместима с библиотекой	<code>pip install --upgrade python-telegram-bot</code> , либо <code>venv</code> на Python 3.12
2	Переключатель "Automatically assign public IPv4 address" не включается при создании сервера	Известный глюк интерфейса Oracle Cloud	Создать сервер без IP, добавить публичный IP отдельно через вкладку "Сети" уже созданного инстанса
3	UNPROTECTED PRIVATE KEY FILE при SSH с Windows	Слишком открытые NTFS-права на файл ключа	<code>icacls</code> — убрать все права кроме своей учётной записи (см. раздел 9)
4	<code>E: Unable to locate package python3.11</code> из <code>deadsnakes PPA</code>	Повреждённый/недокачанный кэш <code>apt</code>	Использовать <code>ruenv</code> вместо PPA (см. раздел 10.2)
5	Процесс компиляции Python обрывается при разрыве SSH	Процесс без защиты завершается вместе с закрытым терминалом	Запускать долгие операции внутри <code>screen</code>
6	<code>screen -r</code> пишет "no screen to be resumed... (Attached)"	Screen-сессия осталась помечена как подключённая к мёртвому терминалу	<code>screen -d -r ИМЯ_СЕССИИ</code> (флаг <code>-d</code> принудительно отключает старое)
7	Синтаксическая ошибка в <code>.bashrc</code> при <code>source ~/.bashrc</code>	Несколько отдельных <code>echo</code> -команд "слиплись" при вставке в терминал	Добавлять переменные одной командой <code>printf</code> , не несколькими <code>echo</code>
8	<code>.env</code> или файл <code>systemd</code> -службы не сохраняется в папку	Терминал закрыт раньше, чем завершена последовательность сохранения	Строго <code>Ctrl+O</code> → <code>Enter</code> → <code>Ctrl+X</code> , затем проверять <code>cat</code>
9	Скрипт сборки указателя по документации (<code>build_manual_index.py</code>) не работает в изолированных ассистент-песочницах	Такие среды часто блокируют исходящий доступ к GitHub/сторонним сайтам	Запускать скрипт именно на сервере — там полноценный интернет
10	<code>git clone</code> зависает на "Username for 'https://github.com'" при сборке указателя	Репозиторий переехал/удалён — GitHub так реагирует и на несуществующий, и на приватный репозиторий	Проверить актуальный адрес в браузере (<code>blender-manual</code> переехал на <code>projects.blender.org</code>)

№	Проблема	Причина	Решение
11	ModuleNotFoundError: No module named 'pkg_resources' при инициализации лемматизатора	руморphy2 (не руморphy3) зависит от setuptools/pkg_resources, которые больше не ставятся в venv по умолчанию на современном Python	Использовать руморphy3 (актуальный форк) — уже прописан в requirements.txt, руморphy2 в проекте не используется
12	tar: options were used after any non-optional arguments при бэкапе перед обновлением	Флаги --exclude были указаны ПОСЛЕ позиционных аргументов (путь к архиву, директория)	--exclude всегда до позиционных аргументов, см. раздел 13.1
13	Первый старт бота стал заметно дольше после добавления лемматизации	руморphy3 грузит словарь русского языка в память при первом обращении к TF-IDF индексу	Ожидаемо (несколько секунд на слабом сервере), не ошибка; RestartSec=5 в systemd не создаёт проблем при таких задержках

Составлено по итогам реального разворачивания и обновления Blender Helper на Oracle Cloud Free Tier. Все описанные проблемы — не гипотетические, а реально встреченные и решённые в процессе. Актуальное состояние проекта, журнал фаз и честный список известных ограничений — PROJECT_PLAN.md. Архитектурные правила — CLAUDE.md. Полное ТЗ — docs/Blender_Expert_System_v2_TZ.pdf.